

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЕТ
по лабораторной работе №1
по дисциплине

**СРЕДСТВА И МЕТОДЫ ЗАЩИТЫ ИНФОРМАЦИИ В
ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ**

Выполнили
студенты гр. 421702

Евик А.Н., Лойко С.П.

Проверил

Сальников Д.А.

Минск 2026

Тема: Генерация паролей

Цель работы

Изучить методы генерации паролей и оценить стойкость пароля к атаке методом прямого перебора. Разработать программу, которая генерирует строку случайных символов заданного алфавита, проверяет равномерность распределения символов в этой строке и вычисляет среднее время подбора пароля, выбранного из сгенерированной строки.

Задание для самостоятельного выполнения

1. Разработать программу, реализующую следующие функции:

- генерация строки с заданной пользователем длиной, состоящей из символов алфавита в соответствии с вариантом задания (использовать инициализацию от таймера);
- проверка равномерности распределения символов путем визуализации частотного распределения;
- вычисление среднего времени подбора пароля, выбираемого из сгенерированной строки.

2. Построить график зависимости среднего времени подбора пароля от его длины.

3. Дать практические рекомендации по выбору пароля исходя из предположений об алфавите пароля; ценности информации, доступ к которой защищается с помощью этого пароля; производительности вычислительного средства атакующего и времени атаки.

Вариант 10: все символы таблицы ASCII.

Ход работы

Выбор алфавита.

Таблица ASCII содержит 128 символов с кодами от 0 до 127. Символы с кодами 0 - 31 и символ с кодом 127 являются управляющими (перевод строки, возврат каретки, табуляция, звонок): они не отображаются на экране и не могут быть введены с клавиатуры в поле пароля. Поэтому в качестве алфавита выбрана печатаемая часть таблицы - символы с кодами от 32 до 127, всего $A = 95$ символов: пробел, знаки препинания, арабские цифры, строчные и прописные буквы латинского алфавита.

В программе алфавит задается одной строкой:

```
ALPHABET = ''.join(chr(code) for code in range(32, 127))
```

Средства реализации.

Программа реализована на языке Python с использованием библиотеки matplotlib для построения графиков.

Расчетные соотношения.

Число всех возможных паролей длины L , составленных из алфавита мощности A , равно $N = A^L$. При переборе в лексикографическом порядке искомый пароль в среднем обнаруживается на середине перебора, то есть после $N/2$ проверок. Отсюда среднее время подбора пароля:

$$T = A^L / (2 \cdot R),$$

где T - среднее время подбора пароля, с; A - мощность алфавита (для варианта 10 $A = 95$); L - длина пароля, символов; R - скорость подбора, то есть количество паролей, проверяемых атакующим за секунду.

Скорость R определяется экспериментально. Для этого программа подбирает пароль "~~~", состоящий из трех последних символов алфавита: в порядке перебора он стоит последним, поэтому проверяются все $95^3 = 857\,375$ вариантов, и скорость вычисляется как отношение числа вариантов к затраченному времени. Полученное значение R используется затем для расчета времени подбора паролей любой длины, перебрать которые в реальном времени невозможно.

Основные функции программы.

1. generate_string(length)

- Назначение: генерирует строку заданной длины из символов алфавита варианта.
- Параметры: length (int) - длина строки, задается пользователем.
- Возвращаемое значение: str - сгенерированная строка.
- Описание: инициализирует генератор псевдослучайных чисел текущим значением системного таймера, после чего length раз выбирает случайный символ из алфавита и объединяет выбранные символы в строку.

2. plot_frequency(text, filename)

- Назначение: строит частотное распределение символов и сохраняет его в файл изображения.

- Параметры: text (str) - исследуемая строка; filename (str) - имя файла для графика.
- Возвращаемое значение: нет.
- Описание: подсчитывает число вхождений каждого символа алфавита в строку, вычисляет ожидаемую частоту как отношение длины строки к мощности алфавита, строит столбчатую диаграмму (по горизонтальной оси - код символа в таблице ASCII, по вертикальной - частота) и наносит на нее красную штриховую линию ожидаемой частоты.

3. crack(password)

- Назначение: подбирает заданный пароль методом прямого перебора и измеряет затраченное время.
- Параметры: password (str) - искомый пароль.
- Возвращаемое значение: float - время подбора в секундах.
- Описание: функция itertools.product последовательно порождает все комбинации символов алфавита длиной len(password) в лексикографическом порядке; каждая комбинация сравнивается с искомым паролем, перебор прекращается при совпадении. Время измеряется от начала перебора до момента совпадения.

4. average_time(password_length, speed)

- Назначение: вычисляет среднее время подбора пароля заданной длины.
- Параметры: password_length (int) - длина пароля; speed (float) - скорость подбора, паролей в секунду.
- Возвращаемое значение: float - среднее время подбора в секундах.
- Описание: реализует расчетную формулу $T = A^L / (2 \cdot R)$.

5. plot_average_time(speed, max_length, filename)

- Назначение: строит график зависимости среднего времени подбора пароля от его длины.
- Параметры: speed (float) - скорость подбора; max_length (int) - максимальная длина пароля, по умолчанию 16; filename (str) - имя файла для графика.
- Возвращаемое значение: нет.

- Описание: для каждой длины от 1 до max_length вычисляет среднее время подбора и строит график. По вертикальной оси используется логарифмическая шкала, поскольку при увеличении длины пароля на один символ время подбора возрастает в 95 раз, и в линейном масштабе кривая нечитаема.

Листинг программы

```
import random

import time

import itertools

import matplotlib.pyplot as plt

ALPHABET = ''.join(chr(code) for code in range(32, 127))

PASS_LENGTH = 3

def generate_string(length: int) -> str:
    random.seed(time.time())
    return ''.join(random.choice(ALPHABET) for _ in range(length))

def plot_frequency(text: str, filename: str="frequency.png"):
    codes = [ord(symbol) for symbol in ALPHABET]
    counts = [text.count(symbol) for symbol in ALPHABET]
    expected = len(text) / len(ALPHABET)
    plt.figure(figsize=(12, 5))
    plt.bar(codes, counts)
    plt.axhline(expected, color='red', linestyle='--',
label='Expected frequency')
    plt.ylim(0, max(counts) * 1.3)
    plt.xlabel('ASCII code')
    plt.ylabel('Frequency')
    plt.title('Symbol frequency in the generated string')
```

```
plt.legend()

plt.savefig(filename, dpi=120)

plt.close()
```

```
def crack(password: str) -> float:

    start_time = time.time()

    for candidate in itertools.product(ALPHABET,
repeat=len(password)):

        if ''.join(candidate) == password:

            break

    return time.time() - start_time
```

```
def average_time(password_length: int, speed: float):

    return len(ALPHABET) ** password_length / 2 / speed
```

```
def plot_average_time(speed, max_length=16,
filename="bruteforce_time.png"):

    lengths = list(range(1, max_length + 1))

    times = [average_time(length, speed) for length in lengths]

    plt.figure(figsize=(10, 6))

    plt.plot(lengths, times, marker='o')

    plt.yscale('log')

    plt.xticks(lengths)

    plt.xlabel('Password length')

    plt.ylabel('Average brute force time, seconds')

    plt.title('Average brute force time vs password length')

    plt.grid(alpha=0.3)

    plt.savefig(filename, dpi=120)
```

```

plt.close()

length = int(input('String length: '))

text = generate_string(length)

plot_frequency(text)

counts = [text.count(symbol) for symbol in ALPHABET]

expected = len(text) / len(ALPHABET)

print(f'Expected: {expected:.1f} | min: {min(counts)} | max: {max(counts)}')

password = text[:PASS_LENGTH]

spent_time = crack(password)

print(f'Password: {password} | cracked in {spent_time:.3f} s')

speed = len(ALPHABET) ** PASS_LENGTH / crack(ALPHABET[-1] * PASS_LENGTH)

print(f'Speed: {speed:.0f} passwords per second')

for password_length in range(1, 17):

    print(f'Length {password_length:2d}: {average_time(password_length, speed):.3e} seconds')

plot_average_time(speed)

```

Результат программы

Задание 1.1. Генерация строки.

При запуске программа запрашивает длину строки. Была задана длина 50 000 символов - такая длина позволяет наглядно проверить равномерность распределения, так как на каждый из 95 символов алфавита приходится в среднем около 526 вхождений.

```

String length: 50000
Expected: 526.3 | min: 472 | max: 581
Password: F{& | cracked in 0.029 s
Speed: 11702522 passwords per second

```

```
String length: 50000
Expected: 526.3 | min: 479 | max: 599
Password: o|86 | cracked in 5.320 s
Speed: 13287307 passwords per second
```

```
String length: 50000
Expected: 526.3 | min: 474 | max: 570
Password: 0"7=" | cracked in 146.611 s
Speed: 11518624 passwords per second
```

Рисунок 1. Генерация строки, подбор пароля и определение скорости перебора для 3, 4 и 5 СИМВОЛОВ

Задание 1.2. Проверка равномерности распределения символов.

Ожидаемая частота каждого символа составляет $50000 / 95 = 526.3$. Фактически самый редкий символ встретился 477 раз, самый частый - 576 раз, наибольшее отклонение от ожидаемого значения составило 50 вхождений (9.4 %). Среднеквадратическое отклонение для такого распределения равно квадратному корню из ожидаемой частоты и составляет 22.8, то есть наибольшее отклонение не выходит за границу трех среднеквадратических отклонений (68.5). Следовательно, все символы алфавита встречаются одинаково часто, а расхождения объясняются случайным характером генерации. На графике это видно по тому, что все столбцы имеют примерно одинаковую высоту и лежат вблизи красной линии ожидаемой частоты.

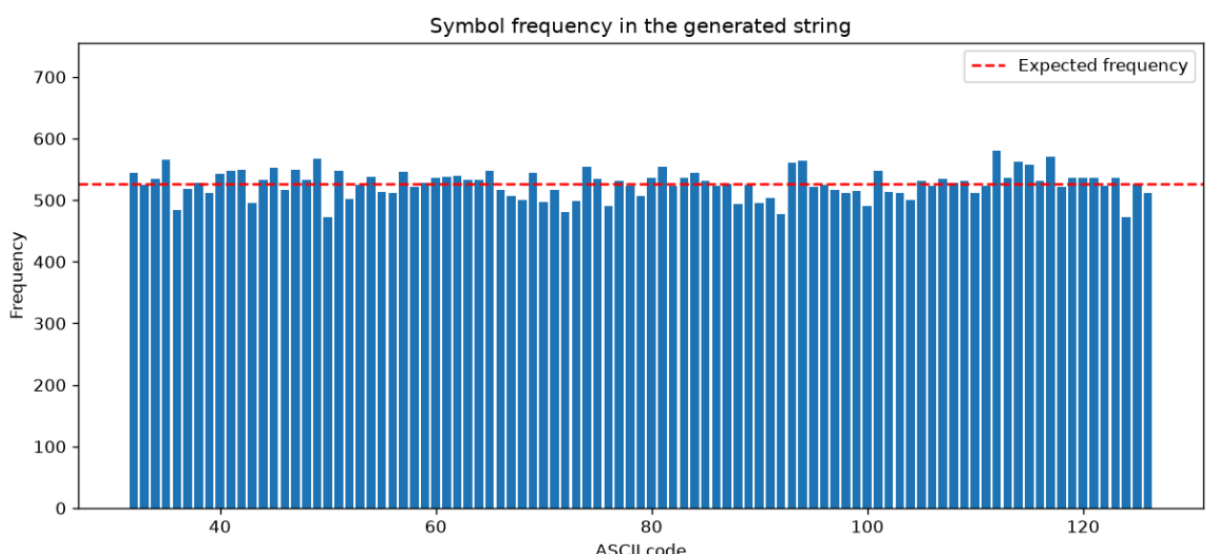
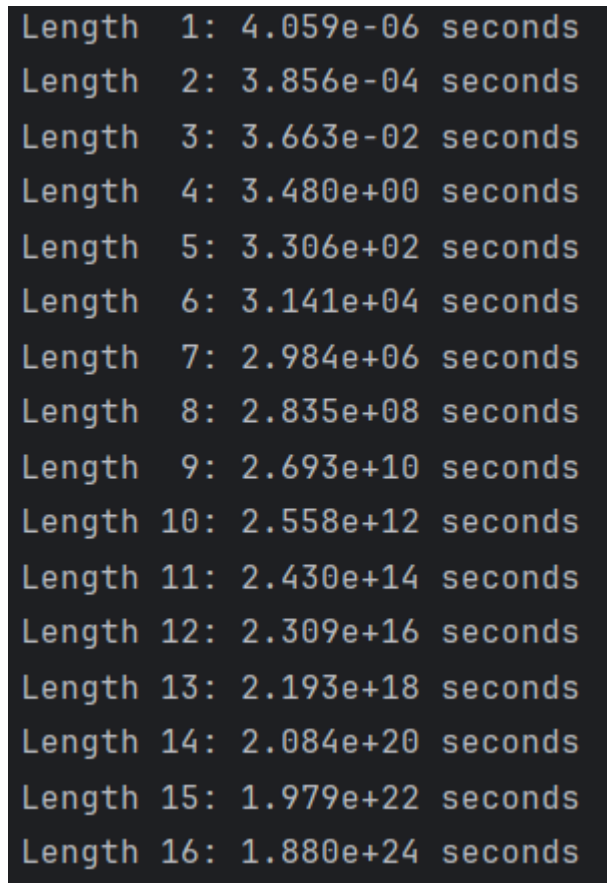


Рисунок 2. График частотного распределения символов в строке длиной 50 000 символов

Задание 1.3. Вычисление среднего времени подбора пароля.

В качестве пароля взяты первые три символа сгенерированной строки - пароль "F{&". Методом прямого перебора он был подобран за 0.029 секунд (рисунок 1). Измеренная скорость подбора составила $R = 11702522$ паролей в секунду. По формуле $T = 95^L / (2 \cdot R)$ вычислено среднее время подбора для паролей длиной от 1 до 16 символов.



Length	1:	4.059e-06	seconds
Length	2:	3.856e-04	seconds
Length	3:	3.663e-02	seconds
Length	4:	3.480e+00	seconds
Length	5:	3.306e+02	seconds
Length	6:	3.141e+04	seconds
Length	7:	2.984e+06	seconds
Length	8:	2.835e+08	seconds
Length	9:	2.693e+10	seconds
Length	10:	2.558e+12	seconds
Length	11:	2.430e+14	seconds
Length	12:	2.309e+16	seconds
Length	13:	2.193e+18	seconds
Length	14:	2.084e+20	seconds
Length	15:	1.979e+22	seconds
Length	16:	1.880e+24	seconds

Рисунок 3. Среднее время подбора пароля в зависимости от его длины

Для 4 символов время подбора пароля составило ~ 5.32 секунд, для 5 символов ~ 146.61 секунд.

Задание 2. График зависимости среднего времени подбора от длины пароля.

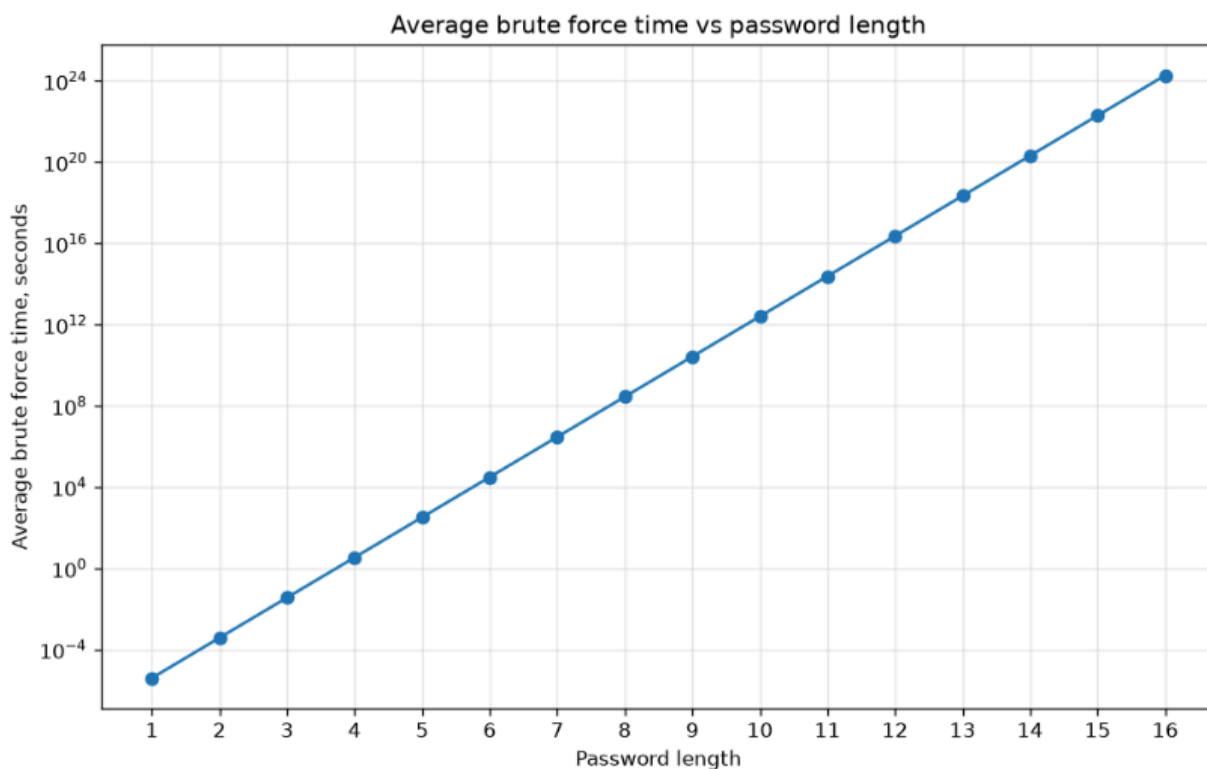


Рисунок 4. Зависимость среднего времени подбора пароля от его длины

График показывает экспоненциальный характер зависимости: увеличение длины пароля на один символ увеличивает время подбора в 95 раз. Пароль из 4 символов подбирается за 5.32 с, из 8 символов - за $2.835 \cdot 10^8$ с (около 7.5 лет), из 12 символов - за $2.309 \cdot 10^{16}$ с (около 500 миллионов лет). Таким образом, увеличение длины пароля является наиболее эффективным способом повышения его стойкости.

Задание 3. Практические рекомендации по выбору пароля.

- Длина пароля важнее используемого набора символов. Каждый добавленный символ увеличивает время подбора в А раз, поэтому длинный пароль из простого алфавита стойче короткого пароля со специальными символами.
- Минимальная рекомендуемая длина пароля из символов ASCII - 12 символов; для защиты особо ценной информации - 16 символов и более.
- Приведенные расчеты справедливы только для случайного пароля. Пароль, придуманный человеком (имя, дата рождения, слово естественного языка), вскрывается словарной атакой за минуты независимо от его длины; замены символов вида "a" на "@" или "o" на "0" учтены в современных словарях и стойкости не добавляют.

- Пароль следует генерировать программно, а не придумывать, и использовать разные пароли для разных ресурсов: утечка базы одного сервиса не должна компрометировать остальные учетные записи.
- Для повышения защищенности следует применять двухфакторную аутентификацию, при которой знание пароля не дает доступа без второго фактора.
- Со стороны защищаемой системы необходимо хранить не сами пароли, а их хэш-значения с солью, вычисляемые криптографически стойкой медленной функцией, а также ограничивать число попыток ввода. Это снижает скорость подбора на несколько порядков и эквивалентно увеличению длины каждого пароля на 2–3 символа.

Вывод

В ходе выполнения лабораторной работы была разработана программа, генерирующая строку заданной пользователем длины из символов таблицы ASCII. Генератор псевдослучайных чисел инициализируется от системного таймера.

Проверка равномерности распределения символов методом визуализации частотного распределения показала, что при длине строки 50 000 символов каждый символ алфавита встречается в среднем 526 раз, а наибольшее отклонение от ожидаемого значения - 54.7 вхождений, что не превышает трех среднеквадратических отклонений. Это подтверждает равномерность работы генератора.

Экспериментально измерена скорость прямого перебора паролей, составившая около $1,7 \cdot 10^7$ паролей в секунду, и на ее основе по формуле $T = A^L / (2 \cdot R)$ вычислено среднее время подбора пароля для длин от 1 до 16 символов. Построенный график показывает экспоненциальный рост времени подбора: увеличение длины пароля на один символ увеличивает время подбора в 95 раз. Пароль из 4 символов подбирается за секунды, из 8 символов - за годы, из 12 символов - за сотни миллионов лет.

На основании расчетов сформулированы практические рекомендации по выбору пароля с учетом мощности алфавита, ценности защищаемой информации, производительности вычислительного средства атакующего и допустимого времени атаки. Основной вывод состоит в том, что длина пароля влияет на его стойкость значительно сильнее, чем расширение используемого алфавита, а расчеты стойкости справедливы только для паролей, сгенерированных случайным образом.